

Optimal foraging algorithm with direction prediction

ZhongQuan Jian, GuangYu Zhu*

College of Mechanical Engineering and Automation, Fuzhou University, Fuzhou, Fujian 35002, PR China

ARTICLE INFO

Article history:

Received 12 August 2019
Received in revised form 25 May 2021
Accepted 27 June 2021
Available online 7 July 2021

Keywords:

Optimal foraging algorithm (OFA)
Direction prediction
Gaussian oscillation
CEC

ABSTRACT

Optimal foraging algorithm (OFA) was presented as a stochastic search algorithm to solve global optimization problems in 2017. As an emerging algorithm, there are many excellent potentials to be explored. To enhance the performance of OFA, a novel optimal foraging algorithm with direction prediction is presented in this paper, named OFA/DP. During the iterations, the population information can be fully utilized by OFA/DP. Once a new optimal solution is found, the evolutionary direction prediction strategy is applied to generate more potential candidates. In addition, considering the situation that the population does not evolve for a long time, which means that the algorithm has achieved the global optima or trapped into local optima. In this case, the Gaussian oscillation strategy is adopted to attempt to find a better solution, and escaping from local optima. To validate the efficiency of the proposed algorithm, the numerical experiments on 20 benchmark functions, 30 CEC test functions, 6 large scale functions, 28 CEC2017 constrained problems, 3 engineering problems, 6 unconstrained multi-objective functions and 10 constrained multi-objective functions are executed. The simulation results and the statistical test demonstrate that OFA/DP has a superior performance in most of functions with faster convergence speed.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

In the last years, a variety of optimization algorithms, inspired by nature analogy, have been proposed in the literature [1], such as genetic algorithm (GA) [2], particle swarm optimization (PSO) [3], ant colony optimization (ACO) [4], simulated annealing (SA) [5], bat algorithm (BA) [6], gray wolf optimizer (GWO) [7], differential evolution (DE) [8] and optimal foraging algorithm (OFA) [9]. These kind of optimization algorithms are also called general algorithms because of their good performance, even in the harsh environment. It is worth mentioning that different algorithms have different properties and performances on different problems, and there is no universal algorithm that can solve all optimization problems well. In reality, there are some complex problems cannot be solved well until now, which need to design special improved algorithms to solve them.

To solve more and more complex practical problems, researchers devote to improving the basic optimization algorithms and make them more robust. In terms of algorithm improvement, researchers focus on finding a careful balance between two conflicting objectives, exploitation and exploration. Exploration is essential to ensure that every part of the solutions domain is searched enough to provide a reliable estimate of the global optimum; exploitation, on the contrary, makes effort around the

best solutions found so far by searching their neighborhoods to reach a better solution.

Generally speaking, there are two approaches to balance exploitation and exploration in the literature. One is so-called hybrid approach, two or more basic algorithms with different advantages are combined to form a new algorithm. Comparing with the basic algorithm, the improved algorithm enhances the advantages of the basic algorithm and makes up its disadvantages at the same time. For example, Liu et al. proposed a novel hybrid algorithm, named PSO-DE. DE was incorporated into update the previous best positions of particles to force PSO jump out of stagnation, because of the strong searching ability of DE [10]; For the purpose of enhancing the search ability of the cuckoo search (CS), Wang et al. added the pitch adjustment operation in harmony search (HS) to the process of the CS updating, so as to speed up convergence [11]. The other approach is to add one or more suitable evolutionary strategies to improve the defect of the algorithm so as to enhance its performance. For instance, Rizk-Allah hybridized sine cosine algorithm (SCA) with multi-orthogonal search strategy to eliminate the flaws of SCA, like unbalanced exploitation and the trapping in local optimal [12]. Lu et al. proposed GWO with cellular topological structure, called CGWO, which can enhance the performance of GWO in exploitation and exploration [13].

In this paper, a novel algorithm named OFA/DP is formed by adding two effective evolutionary strategies to OFA. The evolutionary direction prediction (EDP) strategy, based on the transition probability matrix [14], can make the algorithm get better

* Corresponding author.

E-mail address: zhugy@fzu.edu.cn (G. Zhu).

solutions with high accuracy and speed up its convergence. During the iterations, a new EDP will be established when a new optimal solution is found, and the new EDP will guide the generation of candidates in the next iteration. On the other hand, if the current optimal solution does not update for a long time, which means the global optimal solution has been found or the algorithm is trapped in the local optima. If that happens, the Gaussian oscillation (GO) strategy is employed in the algorithm for enhancing the ability of escaping from the local optima.

The main work and contributions of this paper are summarized as follows:

- During iterations, the population is divided into three states: evolution, exploration and stop, different candidate generation methods are adopted according to the population state.
- Two evolutionary strategies are proposed: the EDP can improve the algorithm's evolutionary efficiency; the GO can enhance the exploration ability of the algorithm.
- The superiority of OFA/DP is verified on 20 benchmark functions, 30 CEC test functions, 6 large scale functions, 28 CEC2017 constrained problems, 3 engineering problems, 6 unconstrained multi-objective functions and 10 constrained multi-objective functions.

The rest of the article will be organized as follows: the basic OFA algorithm will be described detail in Section 2. In Section 3, OFA/DP is proposed. The experiment study of the unconstrained functions, the experiment study of the constrained functions and the experiment study of the multi-objective functions are carried out in Section 4, Section 5 and Section 6, respectively. The sensitivity analysis is executed in Section 7. And Section 8 concludes the article.

2. Optimal foraging algorithm

Without loss of generality, the paradigm of single objective optimization problem (minimum problem) can be expressed as:

$$\text{Minimize } f(\mathbf{x}) \quad (1)$$

where $\mathbf{x} = [x_1, x_2, \dots, x_D]$ represents a D -dimensional vector, and x_i subject to $[x_i^L, x_i^U]$, x_i^L, x_i^U represent the values of lower limit and upper limit of x_i , respectively. $f(\mathbf{x}^*)$ means the objective function of optimal solution with $\mathbf{x}^*$ known as the optimal vector or the best solution.

For solving the unconstrained single objective optimization problem, the standard OFA can be summarized as:

(1) Initialization

- In the D -dimensional vector space, the initial population $\mathbf{G}^t (t = 0)$ is generated according to the following equation:

$$x_i^{j,t} = x_i^L + r * (x_i^U - x_i^L); j = 1, 2, \dots, N; i = 1, 2, \dots, D \quad (2)$$

where r is a random number, uniformly distributed in $(0, 1)$. N is the number of the individuals in the population.

- Evaluating the fitness $f^{j,t}$ of each individual $\mathbf{x}^{j,t}$ in the group $\mathbf{G}^t$.
- Sorting the group $\mathbf{G}^t$ according to the fitnesses f^t from best to worst.
- Setting $f_{best} = f^{1,t}$, $\mathbf{x}_{best} = \mathbf{x}^{1,t}$.

(2) Iteration

- In each iteration, the candidates are generated by the following equations:

$$\begin{cases} x_i^{j,t+1} = x_i^{j,t} + k * r_1 * (x_i^{N,t} - x_i^{j,t}) - k * r_2 * (x_i^{N,t} - x_i^{j,t}); \\ \text{if } j = 1 \\ x_i^{j,t+1} = x_i^{j,t} + k * r_1 * (x_i^{p,t} - x_i^{j,t}) - k * r_2 * (x_i^{p,t} - x_i^{j,t}); \\ \text{if } j \neq 1 \end{cases} \quad (3)$$

where $x_i^{j,t+1}$ represents the j th candidate in iteration t . r_1, r_2 are different random numbers, uniformly distributed in $(0, 1)$. $k = t/t_{max}$ is the scale factor, and t_{max} represents the maximum number of iterations. p is a random integer between 1 and $j - 1$, hence, $x_i^{p,t}$ is better than $x_i^{j,t}$ due to the population $\mathbf{G}^t$ is ordered.

- Inspecting the boundary limits based on the following equation:

$$\begin{cases} x_i^{j,t+1} = 2 * x_i^L - x_i^{j,t+1}; \text{ if } x_i^{j,t+1} < x_i^L \\ x_i^{j,t+1} = 2 * x_i^U - x_i^{j,t+1}; \text{ if } x_i^{j,t+1} > x_i^U \end{cases} \quad (4)$$

- Evaluating the fitness $f^{j,t+1}$ of each individual $\mathbf{x}^{j,t+1}$.
- Checking the candidates and the target individuals by Krebs's model [15] to determine which individual will be kept for the next generation:

$$\frac{\lambda f^{j,t+1}}{1 + (t + 1)\lambda} < \frac{f^{j,t}}{t} \quad (5)$$

where λ is a random number, uniformly distributed in $(0, 1)$. If Eq. (5) is satisfied, setting $\mathbf{x}^{j,t+1} = \mathbf{x}^{j,t+1}$, $f^{j,t+1} = f^{j,t+1}$; otherwise, setting $\mathbf{x}^{j,t+1} = \mathbf{x}^{j,t}$, $f^{j,t+1} = f^{j,t}$.

- Sorting the group $\mathbf{G}^{t+1}$ according to the fitnesses f^{t+1} from best to worst.
- Setting $t = t + 1$, if $f^{1,t} < f_{best}$, setting $f_{best} = f^{1,t}$, $\mathbf{x}_{best} = \mathbf{x}^{1,t}$.

(3) Output

- Repeating the **Iteration** until termination condition is meet.
- Terminating the iteration, and obtain $\mathbf{x}_{best}, f_{best}$ as the final output.

3. Proposed algorithm

In this section, OFA is improved by adding two strategies, named OFA/DP. OFA/DP divides the population into three states, each of which adopts the corresponding candidate generation method, they are as follows:

- The population state is evolution state if the current optimal solution is updated, and the candidates will be obtained according to the EDP.
- The population state is exploration state if the current optimal solution is not updated, and the candidate generation policy complies with the rules of OFA.
- The population state is stop state if the current optimal solution is not updated for a long time, and the candidate generation method is GO.

3.1. The evolutionary direction prediction strategy

There is abundant evolutionary information in the population when the population state is evolution state. In this section, how to mine the evolutionary information in the population is introduced in detail, so as to predict the optimal evolutionary direction of the population.

3.1.1. Transition probability matrix

In mathematics, the transition probability matrix is used to describe the transitions of a Markov chain, and each of the entries is a nonnegative real number less than 1, representing a probability [14]. The meaning of the transition probability matrix is described as follows: as shown in formula (6), $\mathbf{P}$ is 1 step transition probability matrix, where each row represents the state at the current moment, and each column represents the state at the next moment. Hence, p_{ab} represents the probability that the current state is a and the next state is b . There are m states in the current moment, every state a possesses n states at the next moment, and satisfying $\sum_{i=1}^n p_{ai} = 1$ [16–18].

$$\mathbf{P} = \begin{bmatrix} p_{11} & \dots & p_{1b} & \dots & p_{1n} \\ \vdots & \dots & \vdots & \dots & \vdots \\ p_{a1} & \dots & p_{ab} & \dots & p_{an} \\ \vdots & \dots & \vdots & \dots & \vdots \\ p_{m1} & \dots & p_{mb} & \dots & p_{mn} \end{bmatrix} \quad (6)$$

3.1.2. Application of the transition probability matrix

Comparing the i th component of the j th target individual $x_i^{j,t}$ with the i th component of the j th candidate $x_i^{j,t+1}$, it can be found $x_i^{j,t}$ has three ways to generate $x_i^{j,t+1}$: increased, decreased and unchanged. Hence, For the current $x_i^{j,t}$ (regarding as a state), there are three states of $x_i^{j,t+1}$, and the transition probability matrix can be expressed as:

$$\mathbf{P}_i^{j,t} = [p_{11}, p_{12}, p_{13}] \quad (7)$$

where p_{11} , p_{12} and p_{13} represent the probability of $x_i^{j,t+1}$ is generated by $x_i^{j,t}$ in the way of increasing, decreasing and unchanged, respectively. Next, how to calculate the values of p_{11} , p_{12} and p_{13} are introduced.

Three individuals are selected when the population state is evolution state, they are the two best individuals and the worst individual. Since the population has been sorted by the fitness values, the two best individuals are: $\mathbf{x}^{1,t}$ and $\mathbf{x}^{2,t}$, and the worst individual is: $\mathbf{x}^{N,t}$. For every target individual $\mathbf{x}^{j,t}$, its evolutionary direction is expected to close to $\mathbf{x}^{1,t}$ and $\mathbf{x}^{2,t}$, and away from $\mathbf{x}^{N,t}$. Hence, for the i th component of the target individual, $\mathbf{P}_i^{j,t}$ is obtained by the following equations:

$$\begin{cases} p_{11} = (g(x_i^{j,t} < x_i^{1,t}) + g(x_i^{j,t} < x_i^{2,t}) + g(x_i^{j,t} > x_i^{N,t})) / 3 \\ p_{12} = (g(x_i^{j,t} > x_i^{1,t}) + g(x_i^{j,t} > x_i^{2,t}) + g(x_i^{j,t} < x_i^{N,t})) / 3 \\ p_{13} = (g(x_i^{j,t} = x_i^{1,t}) + g(x_i^{j,t} = x_i^{2,t}) + g(x_i^{j,t} = x_i^{N,t})) / 3 \end{cases} \quad (8)$$

where $g(\text{condition})$ is defined as:

$$g(\text{condition}) = \begin{cases} 1, & \text{if condition is true} \\ 0, & \text{if condition is false} \end{cases} \quad (9)$$

To maintain the randomness of the algorithm, when one of the probabilities value is 1, the transition probability matrix will be corrected as follows:

- If $p_{11} = 1$, setting $p_{11} = 0.9$, $p_{12} = 0.1$; if $p_{12} = 1$, setting $p_{12} = 0.9$, $p_{11} = 0.1$.
- If $p_{13} = 1$, setting $p_{13} = 0.6$, $p_{11} = 0.2$, $p_{12} = 0.2$.

As mentioned above, the transition probability matrix $\mathbf{P}_i^{j,t} = [p_{11}, p_{12}, p_{13}]$ of $x_i^{j,t}$ is calculated. Hence, the transition probability

matrix $\mathbf{P}^{j,t}$ of $\mathbf{x}^{j,t}$ is expressed as:

$$\mathbf{P}^{j,t} = [\mathbf{P}_1^{j,t}, \mathbf{P}_2^{j,t}, \dots, \mathbf{P}_D^{j,t}] \quad (10)$$

Next, the candidate $\mathbf{x}^{j,t+1}$ is generated by Algorithm 1.

3.1.3. The superiority of the evolutionary direction prediction strategy

As we know, the transition probability matrix indicates the likelihood from one state to another over a given time horizon. For a D -dimension vector in the hyperspace, the number of states is uncountable from any point of view. Hence, it is difficult to use the decision vector to construct the transition probability matrix directly. Therefore, we focus on the i th dimension of the decision vector, which represents a value in the 1-dimension space. From the perspective of quantitative change, there are three directions of the value to be changed: increasing, decreasing and unchanged. In this way, the change directions of the D -dimension decision vector are countable: 3^D .

In this paper, the transition probability matrix is simplified, we focus on the i th dimension of the decision vector $x_i^{j,t}$. According the definition of the transition probability matrix, $x_i^{j,t}$ possesses three states relative to $x_i^{j,t-1}$. Similarly, $x_i^{j,t+1}$ also possesses three state relative to $x_i^{j,t}$. Hence, the parameters m and n in formula (6) are all 3. In fact, the value of $x_i^{j,t}$ is already determined at iteration t , and it makes no sense to consider its states. Thus, the transition probability matrix of $x_i^{j,t}$ represents as a vector: $\mathbf{P}_i^{j,t} = [p_{11}, p_{12}, p_{13}]$.

According to Section 3.1.2, the accuracy of the transition probability matrix $\mathbf{P}_i^{j,t}$ depends on the accuracy of $x_i^{1,t}$, $x_i^{2,t}$ and $x_i^{N,t}$. Actually, we cannot guarantee the accuracy of $x_i^{1,t}$, $x_i^{2,t}$ and $x_i^{N,t}$, we just assume that they are accurate at the iteration of population is updated, because the population at that moment is potential for evolution.

3.2. The Gaussian oscillation strategy

In this section, the case that the current optimal solution does not update for a long time is considered. Firstly, setting a constant $n_{t_max} = 6$, and the population state is considered as stop state when the number of the iterations that the current optimal solution does not update n_c is not less than n_{t_max} . In this case, the GO strategy is employed in a certain probability r_c , and the population state is expected to change to be evolution state. The principle of the GO strategy is defined as: the candidate $\mathbf{x}^{j,t+1}$ will be randomly generated by a Gaussian distribution with $\mathbf{x}^{j,t}$ as the mean value, and the mathematical formula can be expressed as [19]:

$$\begin{cases} \sigma_i^{j,t} = |r_3 * x_i^{best,t} - r_4 * x_i^{j,t}| \\ x_i^{j,t+1} = N(x_i^{j,t}, (\sigma_i^{j,t})^2) \end{cases} \quad (11)$$

where r_3 , r_4 are two different random numbers, uniformly distributed in $(0, 1)$. $x_i^{best,t}$ represents the i -th component of the current best individual. $N(\mu, \sigma^2)$ represents the Gaussian distribution with μ as the mean value and σ^2 as the variance value.

3.3. Flowchart of OFA/DP

The flowchart of OFA/DP is shown in Fig. 1, and OFA/DP is implemented as follows: Firstly, the initial population is generated randomly in the search space, calculating the fitness of each individual in the population, and sorting the individuals

Algorithm 1

Input: the transition probability matrix $P^{j,t}$, the target individual $x^{j,t}$, the recruited individual $x^{b,t}$, the scale factor k , the vector dimension D .

Output: the candidate $x^{j,t+1}$.

```

01: for  $i = 1:D$  do
02:    $r = \text{rand}(0,1)$ ;  $r_1 = \text{rand}(0,1)$ ;  $r_2 = \text{rand}(0,1)$ .
03:   if  $r < P_{i1}^{j,t}$  do
04:      $x_i^{j,t+1} = x_i^{j,t} + k * |r_1 * (x_i^{p,t} - x_i^{j,t}) - r_2 * (x_i^{p,t} - x_i^{j,t})|$ 
05:   else if  $r < (P_{i1}^{j,t} + P_{i2}^{j,t})$  do
06:      $x_i^{j,t+1} = x_i^{j,t} - k * |r_1 * (x_i^{p,t} - x_i^{j,t}) - r_2 * (x_i^{p,t} - x_i^{j,t})|$ 
07:   else
08:      $x_i^{j,t+1} = x_i^{j,t}$ 
09:   end
10: end

```

Table 1
Experimental results of the benchmark functions.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
F1	3.1E-14	1.7E+03	3.3E+01	0.0E+00	0.0E+00	6.5E-137	9.7E-320
F2	1.5E-02	1.0E+01	5.6E-01	0.0E+00	0.0E+00	8.4E-73	5.4E-263
F3	4.0E+04	5.9E+04	5.1E+04	1.6E-125	1.2E-229	1.4E-06	1.4E-08
F4	7.1E+00	7.7E+01	6.4E+01	1.5E-39	1.0E-224	2.0E-05	2.1E-24
F5	5.2E+01	2.9E+06	5.1E+04	2.5E+01	2.6E+01	2.3E+01	1.3E+01
F6	3.6E-02	1.7E+00	3.6E-01	1.8E-04	6.0E-05	3.9E-03	1.3E-03
F7	1.9E+03	1.4E+03	2.2E+03	5.2E+03	6.3E+03	5.1E+03	2.9E+03
F8	1.2E+00	2.0E+02	1.3E+02	0.0E+00	0.0E+00	1.3E+02	0.0E+00
F9	2.0E+01	2.0E+01	2.0E+01	1.2E+01	8.9E-16	1.5E-09	4.4E-15
F10	3.0E-04	1.7E+01	1.3E+00	0.0E+00	0.0E+00	5.8E-04	0.0E+00
F11	9.7E-08	9.7E+06	3.5E+04	1.4E-04	3.2E-02	1.5E-02	3.7E-31
F12	2.6E-10	2.1E+07	1.9E+05	1.0E-01	6.7E-01	5.8E-06	7.5E-03
F13	2.2E-16	2.2E-16	2.2E-16	6.3E-13	2.6E+00	4.0E-02	7.9E-02
F14	8.4E-04	9.0E-04	1.6E-03	2.3E-04	2.9E-03	5.3E-05	6.7E-05
F15	-1.1E-15	-1.1E-15	-1.1E-15	1.2E-10	9.2E-12	2.7E-06	1.5E-06
F16	2.8E-02	2.1E-04	5.7E-05	7.4E-09	1.8E-06	2.1E-10	2.8E-12
F17	1.1E+00	2.0E-01	9.2E-01	4.1E+00	4.1E-01	1.3E-03	9.9E-02
F18	2.1E-01	1.1E-01	1.1E-01	2.1E+00	3.0E-07	9.0E-04	2.5E-02
F19	2.4E-01	1.1E-01	-7.1E-15	1.2E+00	1.1E-01	1.3E-03	4.9E-03
F20	-3.0E-14	-3.0E-14	-3.0E-14	1.5E-09	3.8E-08	2.9E-10	6.5E-10
Num.	4	4	4	4	9	2	5

from best to worst. Secondly, the candidates are generated by the suitable method selected according to the population state. The bounded constraints of the candidates are guaranteed according to formula (5). Thirdly, the fitness values of the candidates are calculated, and the candidates are compared with the individuals in the original population by formula (6), and the individuals in the new population are selected. Fourthly, determining the population state of the new population, if the current optimal solution is updated, the counter n_c is reset to 0 and the transition probability matrix P^t is calculated; otherwise, P^t is reset to $\mathbf{0}$ and n_c is added 1. Finally, repeating the above steps until the iteration termination condition is met, and the global optimal solution is obtained.

4. Experiment study for the unconstrained functions

In Ref. [9], the performance of OFA has been proved superior to real coded genetic algorithms (RCGAs), PSO, BA, bacteria foraging optimization algorithm (BFOA), shuffled frog-leaping algorithm (SFLA). And DE is a formidable competitor, its performance is close to or better than that of OFA on some issues. To further explore the performance of OFA and DE, three deformation

strategies of DE, DE/rand/1, DE/rand/2 and DE/best/2 [20], are selected in this paper for comparison with OFA and OFA/DP. In addition, the compared algorithms also include SCA and GWO. In this section, the compared algorithms are executed on a suit of famous benchmark functions [9,21], some of CEC2013–CEC2019 test functions [22–26] and some large scale functions [27], which are described in the supplementary material.

4.1. Parameters adaptation

During the performance comparison, setting the population size $N = 20$, the maximum iteration number $t_{max} = 20000$. For OFA/DP, setting the scale factor $k = 1$, the selected rate of GO $r_c = 0.2$ and n_{t_max} is set to 5; for DE/rand/1, DE/rand/2 and DE/best/2, setting the scale factor $F = 1$ and the crossover rate $CR = 0.5$. For SCA, GWO and OFA, their control parameters are adaptive. All experiments are conducted in MATLAB R2019a on a 64-bit PC (Intel(R) Core(TM) i7-8700 CPU @ 3.20 GHz RAM 32.00 GB Windows 10).

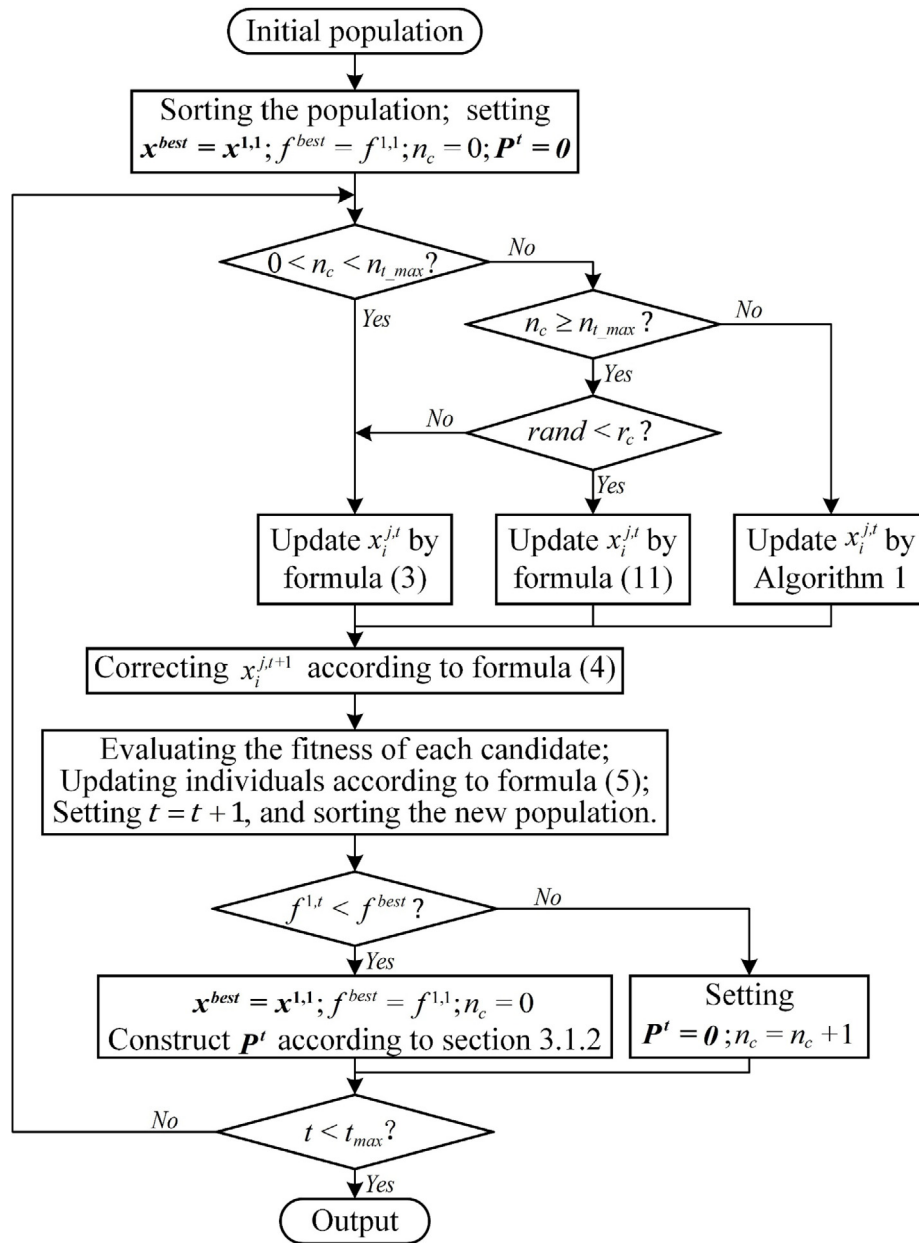


Fig. 1. Flowchart of OFA/DP.

4.2. Result of the benchmark functions

In this section, 20 classical benchmark functions are employed to validate the performance of the proposed algorithm. These benchmark functions are the same as the test functions in [9], where F1–F6 are 30-dimensions unimodal functions, F7–F12 are 30-dimensions multimodal functions and F13–F20 are multimodal functions with fewer numbers of local optima. The detail definitions of these functions are shown in the supplementary material. Each experiment was repeat 50 times with different random seed for the test functions, the mean values of the optimization results of DE/rand/1, DE/rand/2, DE/best/2, SCA, GWO, OFA and OFA/DP obtained in the experiments are given in Table 1, and the best results are typed in bold. In addition, the convergence curves of each algorithm for some of functions are figured in Fig. 2, where the semi-log scale plot is used in drawing the figure, a base 10 logarithmic scale is used for the horizontal axis and a linear scale for the vertical axis.

Table 2
Kruskal–Wallis ANOVA Table.

Source	SS	df	MS	Chi-sq	Prob>Chi-sq
Groups	31725.1	6	5287.51	19.29	0.0037
Error	196840.9	133	1480.01		
Total	228566	139			

From Table 1, it can be found that GWO performs well on the unimodal functions and some multimodal functions, and it achieves the best results on F1, F2, F3, F4, F6, F8, F9, F10 and F18. SCA performs well on F1, F2, F8 and F10. DEs (DE/rand/1, DE/rand/2 and DE/best/2) perform well on the multimodal functions, they outperform other algorithms on F13, F15 and F20. In addition, DE/rand/1 outperforms other algorithms on F12, and DE/rand/2 outperforms other algorithms on F7. Comparing with GWO, OFA/DP is a strong competitor on F1–F10. Comparing with OFA, OFA/DP outperforms OFA on all the benchmark functions

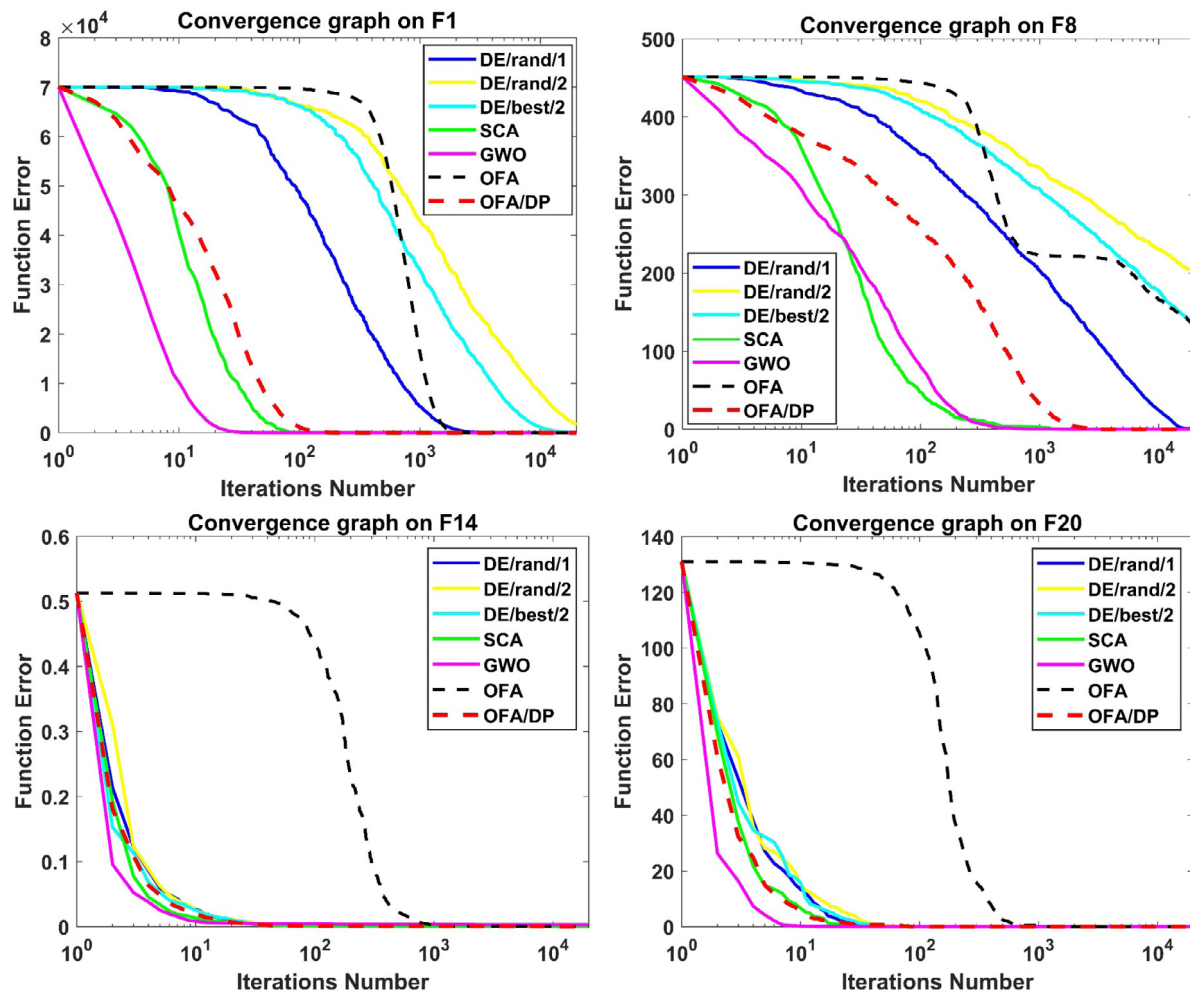


Fig. 2. The evolution curves of the benchmark functions F1, F8, F14 and F20.

Table 3

The estimated values of the mean ranks.

	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
EVMR	74.750	95.275	88.775	59.150	57.100	64.600	53.850
Errors	9.0674	9.0674	9.0674	9.0674	9.0674	9.0674	9.0674

Table 4

Experimental results of the benchmark functions.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
F1	5.992(3)	6.825(6)	6.604(5)	4.381(1)	8.105(7)	5.136(2)	6.513(4)
F2	5.349(3)	6.079(6)	5.933(5)	3.947(1)	7.274(7)	4.625(2)	5.745(4)
F3	6.795(3)	7.578(5)	7.426(4)	5.296(1)	8.846(7)	6.020(2)	7.588(6)
F4	5.655(3)	6.415(6)	6.288(5)	4.134(1)	7.674(7)	4.866(2)	6.216(4)
F5	5.417(3)	6.176(6)	6.105(5)	3.952(1)	7.361(7)	4.682(2)	5.966(4)
F6	6.551(3)	7.361(5)	7.181(4)	4.910(1)	8.572(7)	5.740(2)	7.490(6)
F7	6.107(3)	6.899(5)	6.793(4)	4.586(1)	8.095(7)	5.334(2)	6.945(6)
F8	6.077(3)	6.906(6)	6.753(4)	4.456(1)	7.882(7)	5.332(2)	6.804(5)
F9	6.134(3)	6.908(6)	6.736(4)	4.578(1)	7.946(7)	5.319(2)	6.880(5)
F10	6.158(3)	7.064(6)	6.870(4)	4.591(1)	8.011(7)	5.381(2)	7.020(5)
F11	5.933(3)	7.635(7)	7.314(5)	4.363(1)	7.586(6)	5.253(2)	6.546(4)
F12	5.982(3)	7.795(7)	7.520(5)	4.348(1)	7.549(6)	5.091(2)	6.513(4)
F13	7.114(3)	7.792(5)	7.633(4)	5.073(1)	8.651(7)	6.592(2)	8.062(6)
F14	4.552(3)	5.214(6)	5.073(5)	2.611(1)	6.059(7)	4.062(2)	4.723(4)
F15	4.599(3)	5.276(6)	5.104(4)	2.629(1)	6.098(7)	4.095(2)	5.521(5)
F16	4.468(3)	5.140(5)	4.976(4)	2.489(1)	5.973(7)	3.983(2)	5.404(6)
F17	4.675(3)	5.350(5)	5.197(4)	2.706(1)	6.164(7)	4.129(2)	5.555(6)
F18	4.689(3)	5.370(5)	5.218(4)	2.735(1)	6.184(7)	4.162(2)	5.591(6)
F19	4.734(3)	5.432(5)	5.265(4)	2.776(1)	6.240(7)	4.208(2)	5.632(6)
F20	4.546(3)	5.199(5)	5.042(4)	2.544(1)	6.036(7)	4.045(2)	5.483(6)

Table 5
Summary of the CEC test functions.

No.	Functions	Source	C(x*)
C1	Sphere Function	CEC2013	−1400
C2	Rotated Ackley's Function	CEC2013	−700
C3	Rotated Katsuura Function	CEC2013	200
C4	Composition Function 5	CEC2013	1100
C5	Composition Function 8	CEC2013	1400
C6	Rotated High Conditioned Elliptic Function	CEC2014	100
C7	Shifted and Rotated Rastrigin's Function	CEC2014	900
C8	Hybrid Function 2	CEC2014	1800
C9	Composition Function 3	CEC2014	2500
C10	Composition Function 8	CEC2014	3000
C11	Rotated High Conditioned Elliptic Function	CEC2015	100
C12	Shifted and Rotated Rastrigin's Function	CEC2015	400
C13	Hybrid Function 5	CEC2015	800
C14	Composition Function 4	CEC2015	1200
C15	Composition Function 7	CEC2015	1500
C16	Shifted and Rotated Bent Cigar Function	CEC2017	100
C17	Shifted and Rotated levy Function	CEC2017	900
C18	Hybrid Function 6	CEC2017	1600
C19	Composition Function 4	CEC2017	2400
C20	Composition Function 9	CEC2017	3000
C21	Storn's Chebyshev Polynomial Fitting Problem	CEC2019	1
C22	Inverse Hilbert Matrix Problem	CEC2019	1
C23	Lennard-Jones Minimum Energy Cluster	CEC2019	1
C24	Rastrigin's Function	CEC2019	1
C25	Griewangk's Function	CEC2019	1
C26	Weierstrass Function	CEC2019	1
C27	Modified Schwefel's Function	CEC2019	1
C28	Expanded Schaffer's F6 Function	CEC2019	1
C29	Happy Cat Function	CEC2019	1
C30	Ackley Function	CEC2019	1

except F5, F7, F12, F19 and F20, which indicates that the improvement method in OFA/DP is efficient. Moreover, OFA/DP can obtain the best results on 5 functions, they are F5, F8, F10, F11 and F16.

For the sake of reflect and analyze the quantitative characteristics of things as a whole and observe the nature and development rules of things, a statistical analysis tool, Kruskal–Wallis test [28], was used to collate and observe the experimental data, the statistical result is shown in Table 2.

The p -value ($p = 0.0037$) is less than 0.05, which shows that there exists at least one algorithm significantly better than others when handling these benchmark functions. Then, multiple comparison procedure is carried out to analyze the result of Kruskal–Wallis test, and the estimated values of the mean ranks (EVMR) are achieved, shown in Table 3.

Table 3 indicates that the estimated value of the mean ranks of OFA/DP is the smallest one, which means the performance of OFA/DP is the best when handling these benchmark functions in terms of the accuracy of the results. The second best algorithm is GWO, the third is SCA, and the fourth is OFA.

From Fig. 2, it can be found that the convergence speed of GWO is the fastest on F1, F14 and F20. The convergence speed of OFA is the smallest on F14 and F20. In addition, the convergence speed of OFA/DP is faster than that of OFA on these four functions, which indicates the proposed strategies is effective, and these strategies can not only accelerate the speed of the convergence, but also enhance the convergence performance.

Table 4 summarized the mean of the CPU time consumed for the benchmark functions, numbers in parentheses indicate the rank of the consumed time of the algorithm on the function, the bold font represents the result of the best algorithm on the function (the same as that in Table 1). As shown in Table 4, SCA consumes the least time on all the benchmark functions, and it can obtain the best result on F1, F2, F8 and F10, which indicates that SCA outperforms the other algorithms on these functions. GWO spends the most time on all the benchmark functions except F11 and F12, and it can obtain the best result on F1, F2, F3, F4, F6, F8, F9, F10 and F18. According the no-free-lunch theorems [29], it can be concluded that GWO obtains the

high precision results by consuming a lot of computing resources. Comparing OFA/DP with OFA, it can be found that the running times of OFA/DP is bigger than that of OFA on all the benchmark functions. According to the result of Table 3, OFA/DP is the best algorithm when handling the benchmark functions, it can be concluded that the strategies of OFA/DP increases the computing resources, but more accurate results are obtained.

4.3. Result of the CEC test functions

In this section, 30 CEC functions are selected from CEC2013, CEC2014, CEC2015, CEC2017 and CEC2019 to test the proposed algorithm, these functions contain different characteristics in each test suite. The basic information of C1–C30 is summarized in Table 5, and the detail definitions of these CEC test functions are shown in the supplementary material.

From Table 6, it can be found that DE/rand/1 outperforms other algorithms on C24, DE/best/2 outperforms other algorithms on C2. SCA achieves the best results on C3, C7, C12, C18, C22 and C27. GWO obtains the best results on C21 and C23. OFA performs well on 8 functions, they are C4, C5, C8, C13, C14, C16, C19 and C26. OFA/DP performs well on 12 functions, they are C1, C6, C9, C10, C11, C15, C17, C20, C25, C28, C29 and C30. In terms of the number of best results, OFA/DP is the best algorithm, and OFA is the second best algorithm.

Furthermore, the Kruskal–Wallis test was used to analyze the experimental result, and the statistical result is obtained, shown in Table 7. As shown in Table 7, the p -value ($p = 0.6939$) is greater than 0.05, which shows that there is not an algorithm significantly better than others when handling these CEC test functions.

As shown in Fig. 3, the semi-log scale plot is used in drawing the figure, a base 10 logarithmic scale is used for the horizontal axis and a linear scale for the vertical axis. GWO possesses the fastest convergence rate on C1, C7, C13, C19 and C25 within 100 iterations. However, its convergence performance become worse in the subsequent iterations. From Fig. 3, it can be found that the

Table 6
Experimental results of the CEC functions.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
C1	2.98E+02	4.56E+02	5.05E+02	2.53E+01	1.67E+03	4.3E−01	2.79E−01
C2	20.9296	20.9300	20.9208	20.9368	20.9357	20.9609	20.9582
C3	2.44900	2.40972	2.38855	1.50597	2.47804	2.38740	2.38047
C4	285.880	300.142	286.964	273.695	274.955	250.017	278.242
C5	1.46E+03	1.72E+03	1.94E+03	5.24E+02	1.64E+03	2.8E+02	3.01E+02
C6	4.11E+07	7.14E+07	4.09E+07	1.12E+07	8.03E+07	5.6E+06	3.61E+06
C7	220.680	279.016	261.487	86.1093	98.0960	170.422	178.708
C8	3.39E+07	1.20E+08	1.58E+08	3.82E+04	9.52E+06	1.8E+03	3.46E+03
C9	213.899	217.709	213.137	205.537	210.685	204.611	203.918
C10	1.73E+04	1.87E+04	3.43E+04	9.10E+03	5.07E+04	2.3E+03	2.22E+03
C11	4.59E+07	7.42E+07	6.45E+07	7.20E+06	2.10E+07	2.5E+06	5.31E+05
C12	209.088	260.843	247.136	72.2523	98.6162	178.997	168.481
C13	1.74E+05	6.09E+05	3.62E+05	1.54E+05	3.77E+05	3.4E+04	7.96E+04
C14	191.056	199.064	191.215	183.621	121.574	106.036	107.111
C15	110.873	125.059	133.855	105.274	218.883	101.431	101.285
C16	3.26E+08	1.58E+09	8.41E+08	4.37E+07	2.34E+09	3.3E+03	5.32E+03
C17	4.62E+02	8.30E+03	6.15E+03	2.91E+02	1.14E+03	4.1E+01	5.71E−01
C18	1.07E+03	1.04E+03	1.01E+03	5.06E+02	7.32E+02	1.3E+03	9.37E+02
C19	626.594	653.133	647.530	476.006	543.565	467.658	591.278
C20	1.58E+04	8.23E+04	4.30E+04	6.37E+05	6.91E+06	2.5E+04	5.65E+03
C21	7.74E+05	1.90E+06	6.50E+06	6.13E+03	1.29E+02	2.6E+04	3.73E+04
C22	2.08E+03	3.46E+03	2.55E+03	1.40E+02	1.64E+02	5.5E+02	4.22E+02
C23	2.26268	3.99091	2.22974	1.34169	0.96269	3.09947	1.57966
C24	2.89516	9.99848	5.47086	6.05252	11.4617	22.7035	5.77620
C25	0.11155	0.34640	0.46330	0.23907	0.53423	0.12668	0.01343
C26	1.98644	1.87064	2.43387	1.20497	1.41313	0.43182	0.47773
C27	425.192	358.600	454.823	156.404	521.065	928.692	365.969
C28	2.41131	2.65640	2.69579	1.90611	2.24759	2.33005	1.51811
C29	0.16297	0.22229	0.20826	0.11988	0.10012	0.13084	0.09046
C30	20.0758	20.0784	20.0803	20.1301	19.6718	10.5495	9.08634
Num	1	0	1	6	2	8	12

Table 7
Kruskal-Wallis ANOVA Table.

Source	SS	df	MS	Chi-sq	Prob>Chi-sq
Groups	14300.9	6	2383.49	3.87	0.6939
Error	757431.6	203	3731.19		
Total	771732.5	209			

Table 8
Summary of the large scale functions.

No.	Functions	Categories	$C(\mathbf{x}^*)$
LS1	Elliptic Function	Fully-separable Function	0
LS2	Rastrigin Function 1	Fully-separable Function	0
LS3	Rastrigin Function 2	Functions with a separable subcomponent	0
LS4	Ackley Function	Functions with no separable subcomponent	0
LS5	Rosenbrock's Function	Overlapping Function	0
LS6	Schwefels Problem 1.2	Non-separable Function	0

convergence speed of OFA/DP is faster than OFA on all functions, which indicates that OFA/DP is better than OFA.

4.4. Result of the large scale functions

In this section, seven algorithms are executed in 6 large scale functions to verify their performances. These large scale functions are selected from CEC2013 large scale global optimization problems [27], the basic information of LS1–LS6 is summarized in Table 8, and the detail definitions of these large scale functions are shown in the supplementary material. Each experiment was repeat 25 times with different random seeds, and the mean values of the optimization results are tabulated in Table 9, the best results are typed in bold. In addition, the convergence curves of each algorithm are figured in Fig. 4.

As shown in Table 9, the large scale functions are really a challenge for these algorithms, there is no an algorithm can obtain the

optimal solutions. SCA performs well on LS3, OFA outperforms other algorithms on LS1, LS5 and LS6, and OFA/DP can get the better solutions on LS2 and LS4.

Moreover, the Kruskal-Wallis test was used to analyze the experimental result, and the statistical result is obtained, shown in Table 10.

As shown in Table 10, the p -value ($p = 0.8576$) is greater than 0.05, which shows that there is not an algorithm significantly better than others when handling these large scale functions.

Fig. 4 depicts the evolution curves of the large scale functions, which demonstrates that the convergence speed of GWO is the fastest on all large scale functions except LS4 within 100 iterations, the convergence speed of OFA/DP is faster than OFA on all the large scale functions within 1000 iterations.

5. Experiment study for the constrained functions

Without loss of generality, the paradigm of the constrained optimization problem can be formulated as:

$$\begin{aligned} \text{Minimize} \quad & f(\mathbf{x}), \mathbf{x} = [x_1, x_2, \dots, x_D], x_i^L \leq x_i \leq x_i^U \\ \text{Subject to:} \quad & g_k(\mathbf{x}) \leq 0, k = 1, 2, \dots, p \\ & h_k(\mathbf{x}) = 0, k = p + 1, p + 2, \dots, q \end{aligned} \quad (12)$$

where $g_k(\mathbf{x})$ is the k th inequality constraint, and $h_k(\mathbf{x})$ is the $(k - p)$ -th equality constraint. The equality constraints are usually converted into inequality constraints by a tolerance value δ (usually set to 0.0001), as follows: $|h_k(\mathbf{x})| - \delta \leq 0$. Then, the constraint is converted to:

$$q_k(\mathbf{x}) = \begin{cases} \max(0, g_k(\mathbf{x})), & \text{if } 1 \leq k \leq p \\ \max(0, |h_k(\mathbf{x})| - \delta), & \text{if } p + 1 \leq k \leq q \end{cases} \quad (13)$$

Hence, the constrained problem can be transformed into the unconstrained problem by punishing the constraint items. The mathematical formula is described as follows:

$$\text{Minimize} \quad f(\mathbf{x}) + \rho \sum_{k=1}^q q_k(\mathbf{x}) \quad (14)$$

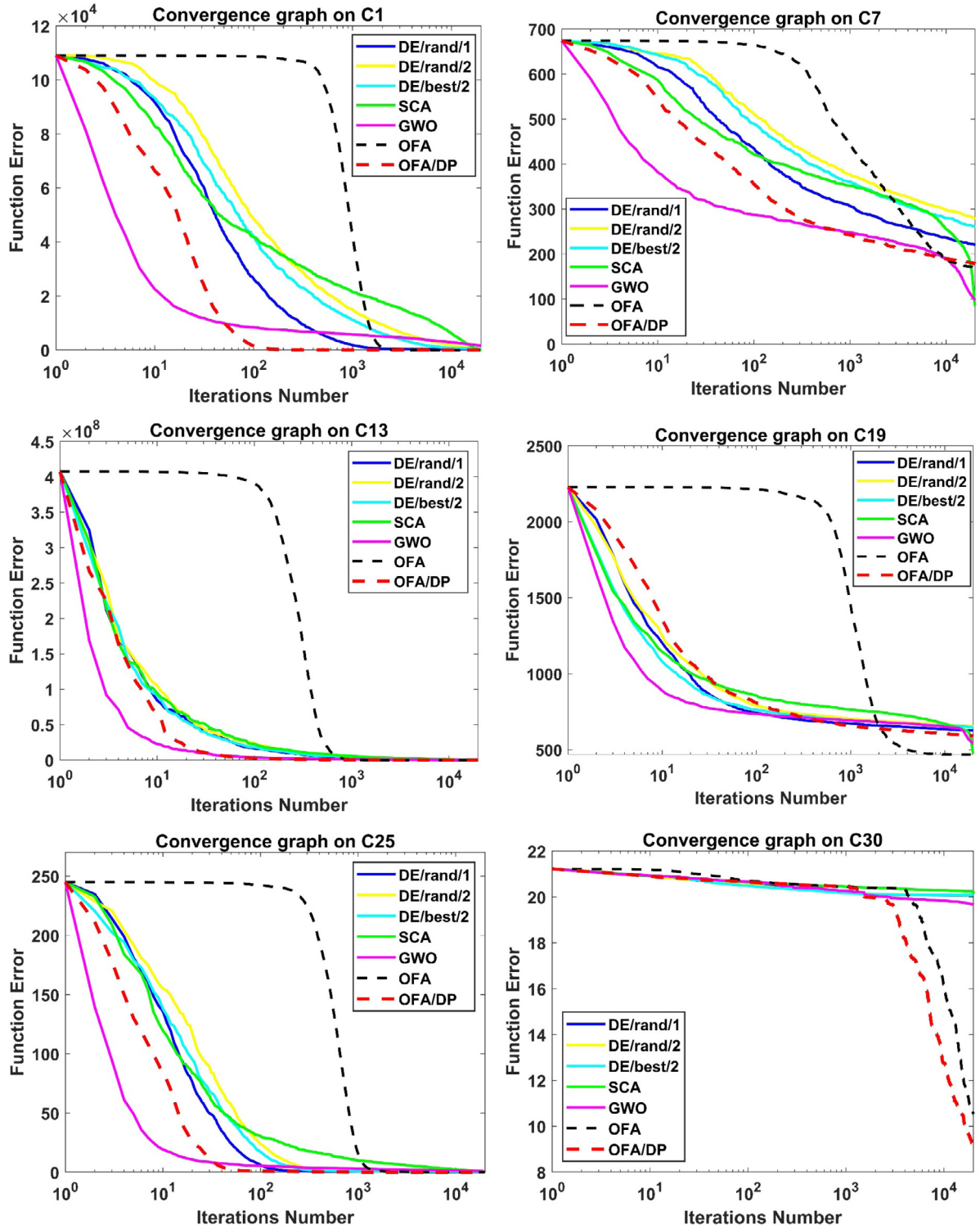


Fig. 3. The evolution curves of the CEC functions C1, C7, C13, C19, C25 and C30.

where ρ denotes the penalty coefficient (set to 10^8), which has no effect when all the constraints are satisfied.

5.1. CEC2017 constrained problems

In this section, numerical experiments are carried out on 28 constraint functions, which are described detailed in [30]. Each experiment was repeat 50 times with different random seed for

the test functions, the mean values of the optimization results are tabulated in Table 11, and the best results are typed in bold.

As shown in Table 11, the numbers of best results that DE/rand/1, DE/rand/2, DE/best/2, SCA, GWO, OFA and OFA/DP can obtain are 2, 0, 1, 3, 2, 6 and 14 respectively. In terms of the number of best results, OFA/DP outperforms other algorithms absolutely. From Table 11, it can be found that DE/rand/1 outperforms other algorithms on CF4 and CF20, DE/best/2 outperforms other algorithms on CF19. SCA achieves the best results on CF6,

Table 9

Experimental results of the large scale functions.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
LS1	3.9E+10	1.0E+11	7.4E+10	9.5E+09	2.8E+10	1.2E+07	1.3E+08
LS2	2.8E+04	5.0E+04	4.2E+04	1.5E+04	1.5E+04	9.1E+03	7.8E+03
LS3	7.5E+06	1.1E+07	9.7E+06	4.3E+06	5.0E+06	5.8E+06	5.0E+06
LS4	4.7E+07	4.7E+07	4.7E+07	4.7E+07	4.7E+07	4.7E+07	4.7E+07
LS5	3.3E+12	4.3E+12	4.3E+12	3.5E+11	5.1E+11	3.6E+10	5.3E+11
LS6	1.8E+14	1.1E+16	3.4E+15	2.7E+12	4.6E+13	8.0E+07	3.3E+10
Num.	0	0	0	1	0	3	2

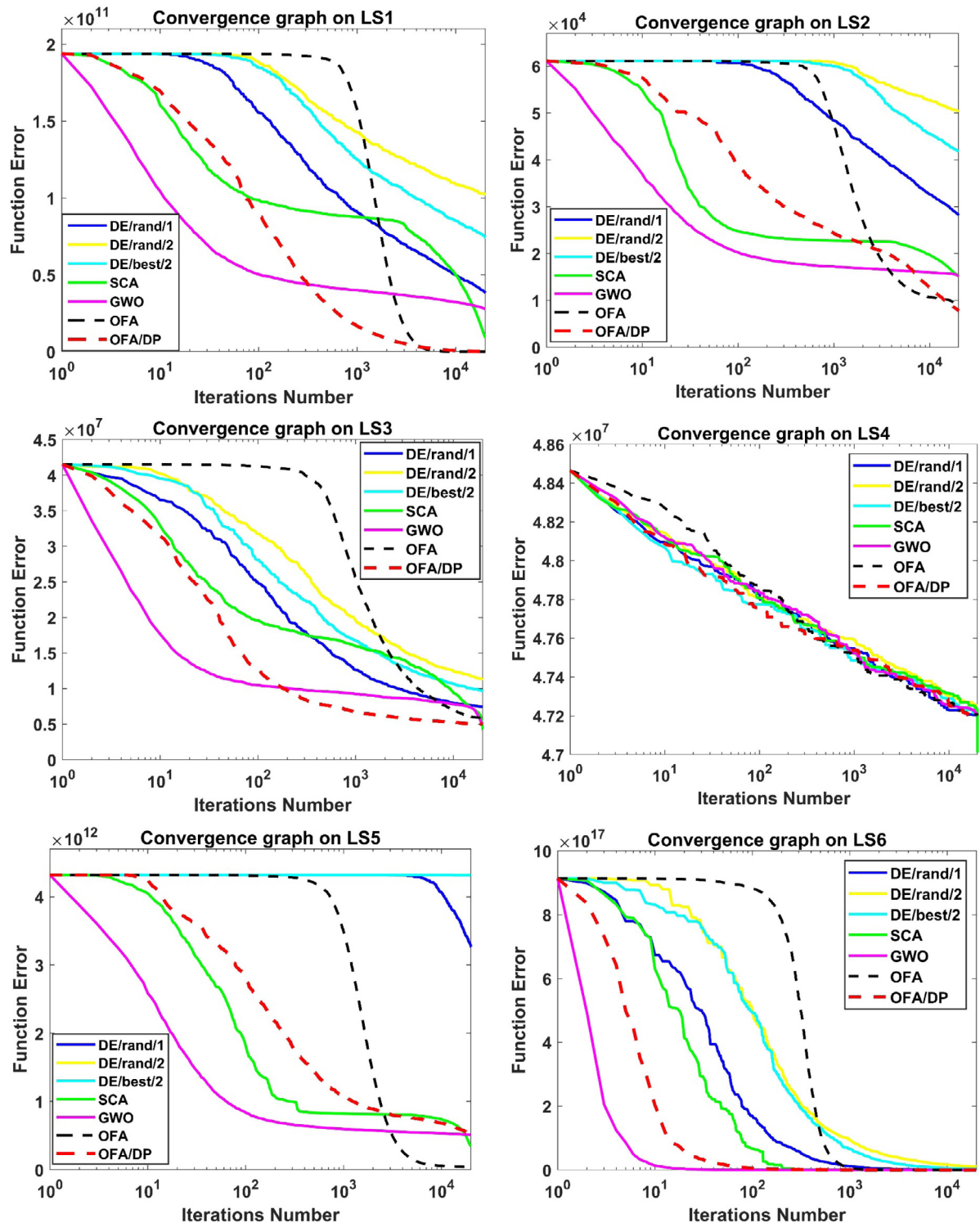
**Fig. 4.** The evolution curves of the large scale functions.

Table 10
Kruskal-Wallis ANOVA Table.

Source	SS	df	MS	Chi-sq	Prob>Chi-sq
Groups	390.67	6	65.111	2.6	0.8576
Error	5779.83	35	165.138		
Total	6170.5	41			

CF7 and C28. GWO obtains the best results on CF18 and CF27. OFA performs well on 6 functions, they are CF9, CF14, CF15, CF17, CF21 and CF26. OFA/DP performs well on CF1, CF2, CF3, CF5, CF8, CF10, CF11, CF12, CF13, CF16, CF22, CF23, CF24 and CF25. In addition, the performances of OFA and OFA/DP are much higher than the other algorithms on CF1, CF2, CF8, CF10, CF12, CF14, CF21 and CF23, which indicates that OFA and OFA/DP have advantages in dealing with the CEC2017 constrained problems.

Furthermore, the Kruskal-Wallis test was used to analyze the experimental result, and the statistical result shown in Table 12. As shown in Table 12, the p -value ($p = 4.5374e - 05$) is less than 0.05, which shows that there exists at least one algorithm significantly better than others when handling these benchmark functions. Then, multiple comparison procedure is executed to analyze the result of Kruskal-Wallis test, and the estimated values of the mean ranks (EVMR) are achieved, shown in Table 13.

Table 13 indicates that the estimated value of the mean ranks of OFA/DP is the smallest one, which means the performance of OFA/DP is the best when handling the CEC2017 constrained problems. The second best algorithm is OFA.

5.2. Engineering problems

To test the generalization ability of the proposed algorithms, numerical experiments are carried out on three practical engineering problems. The engineering problem is essentially a mathematical optimization problem, a function optimization problem with constraints. The detail definitions of these engineering problems are shown in the supplementary material. Each experiment

Table 12
Kruskal-Wallis ANOVA Table.

Source	SS	df	MS	Chi-sq	Prob>Chi-sq
Groups	95474.4	6	15912.4	29.67	4.5374E-05
Error	531970.6	189	2814.7		
Total	627445	195			

was repeat 50 times with different random seeds, and the mean values, the best values, the median values and the worst values of the optimization results are tabulated in Tables 14–16, the best results are typed in bold.

As shown in Table 14, DE/rand/2 and DE/best/2 can converge to the optimum, and they possess the best results of the mean value, the best value, the median value and the worst value. The results of the best value and the median value of DE/rand/1 are the same as DE/rand/2 and DE/best/2. In addition, it can be found that OFA/DP outperforms OFA on the welded beam design problem.

From Table 15, it can be found that all the compared algorithms can convergence to the optimum, and all the compared algorithms can obtain the best results of the median value. GWO, OFA and OFA/DP can achieve the best results of the mean value. OFA and OFA/DP can get the best results of the worst value. As a conclusion, OFA and OFA/DP outperform the other algorithms in term of the accuracy of the results on the three-bar truss design problem.

For the speed reducer design problem, OFA/DE possesses the best results on the mean value, the best value, the median value and the worst value. DE/rand/1, DE/rand/2 and DE/best/2 can get the best results of the best value. Hence, it can be concluded that can OFA/DP performs well on the speed reducer design problem.

6. Experiment study for the multi-objective functions

Generally, the mathematical description of the multi-objective optimization problems (MOP) is:

$$\text{Minimize } f(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})],$$

Table 11
Experimental results of the CEC2017 constrained problems.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
CF1	2.6E+04	4.2E+04	3.7E+04	8.4E+02	2.6E+03	2.2E-06	2.9E-07
CF2	1.3E+04	4.9E+04	2.8E+04	8.7E+02	2.4E+03	2.0E-04	1.9E-07
CF3	7.5E+05	1.3E+06	1.2E+06	5.8E+05	5.0E+05	7.5E+05	4.9E+05
CF4	1.4E+02	3.5E+02	3.1E+02	1.6E+02	1.7E+02	2.7E+02	1.6E+02
CF5	1.7E+03	1.5E+05	9.5E+03	1.1E+02	8.8E+03	2.6E+01	9.0E+00
CF6	1.2E+09	1.3E+09	1.2E+09	1.8E+08	6.2E+08	7.5E+08	1.0E+09
CF7	2.0E+06	2.9E+07	1.1E+07	8.4E+05	6.1E+09	2.8E+09	5.0E+10
CF8	3.6E+12	5.6E+12	4.9E+12	3.9E+09	1.8E+10	3.9E-04	-1.4E-4
CF9	3.0E+09	3.8E+08	5.8E+09	7.1E+06	1.2E+09	1.3E+06	3.8E+06
CF10	1.7E+11	7.5E+12	4.7E+12	5.4E+10	1.4E+12	-8.5E-5	-9.9E-5
CF11	7.3E+11	7.4E+11	7.4E+11	1.4E+09	1.3E+10	1.9E+08	4.1E+07
CF12	7.8E+07	1.2E+11	2.1E+09	4.1E+08	9.1E+10	7.7E+00	7.2E+00
CF13	6.4E+09	1.4E+11	1.9E+10	8.6E+09	1.2E+11	5.6E+09	2.0E+03
CF14	2.2E+00	2.3E+11	4.4E+09	4.1E+09	1.7E+11	2.2E+00	1.9E+02
CF15	8.4E+05	2.0E+06	2.3E+05	1.8E+01	1.1E+09	1.7E+01	1.9E+01
CF16	8.2E+03	6.1E+05	8.9E+04	2.1E+02	1.9E+02	2.1E+02	1.9E+02
CF17	3.4E+09	1.0E+11	5.4E+09	3.1E+09	9.0E+10	3.1E+09	3.1E+09
CF18	2.8E+18	3.6E+18	2.4E+18	7.6E+16	7.7E+15	1.6E+18	5.1E+17
CF19	4.3E+12	4.3E+12	4.3E+12	4.3E+12	4.3E+12	4.3E+12	4.3E+12
CF20	4.9E+00	5.7E+00	5.0E+00	6.4E+00	7.4E+00	7.5E+00	7.7E+00
CF21	1.9E+10	4.3E+12	1.3E+12	9.3E+09	4.7E+11	8.2E+00	1.1E+01
CF22	4.4E+10	4.2E+12	1.3E+12	3.0E+10	5.2E+11	1.7E+10	1.1E+10
CF23	2.9E+09	8.3E+12	2.5E+12	2.7E+10	8.9E+11	2.2E+00	2.2E+00
CF24	1.4E+10	3.8E+12	8.9E+11	2.0E+01	1.2E+11	2.0E+01	1.9E+01
CF25	7.0E+09	3.6E+12	9.0E+11	8.0E+02	1.6E+11	2.1E+02	2.1E+02
CF26	5.8E+09	4.2E+12	1.2E+12	6.5E+09	4.7E+11	3.1E+09	3.1E+09
CF27	2.9E+19	3.9E+19	1.9E+19	1.2E+18	6.9E+16	2.2E+19	8.4E+18
CF28	4.3E+12	4.3E+12	4.3E+12	4.3E+12	4.3E+12	4.3E+12	4.3E+12
Num.	2	0	1	3	2	6	14

Table 13

The estimated values of the mean ranks.

	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
EVMR	103.0357	125.5714	118.4643	92.5357	115.2143	68	66.6786
Errors	10.7199	10.7199	10.7199	10.7199	10.7199	10.7199	10.7199

Table 14

Result of the welded beam design problem.

Cri.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
Mean	1.72486	1.72485	1.72485	1.72581	1.72503	1.73026	1.72694
Best	1.72485	1.72485	1.72485	1.72512	1.72487	1.72486	1.72486
Median	1.72485	1.72485	1.72485	1.72580	1.72498	1.72489	1.72487
Worst	1.72524	1.72485	1.72485	1.72665	1.72574	1.83359	1.75914

Table 15

Result of the three-bar truss design problem.

Cri.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
Mean	264.469	265.133	266.324	268.065	263.896	263.896	263.896
Best	263.896	263.896	263.896	263.896	263.896	263.896	263.896
Median	263.896	263.896	263.896	263.896	263.896	263.896	263.896
Worst	271.493	282.843	282.843	282.843	263.897	263.896	263.896

Table 16

Result of the speed reducer design problem.

Cri.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
Mean	3004.31	2999.16	3009.51	2998.76	2998.34	2996.43	2996.28
Best	2996.23	2996.23	2996.23	2997.08	2996.34	2996.30	2996.23
Median	2997.09	2996.23	2999.86	2998.70	2997.39	2996.34	2996.28
Worst	3035.51	3035.51	3035.51	3001.05	3005.61	3001.08	2996.35

Table 17

Experimental results of the unconstrained multi-objective functions.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
UM1	0.38506	0.38000	0.38759	0.39000	0.37500	0.37523	0.37516
UM2	0.50000	0.50000	0.50000	0.50000	0.50000	0.50032	0.50012
UM3	0.11841	0.09226	0.11859	0.09426	0.03894	0.03894	0.03896
UM4	0.41062	0.39321	0.40292	0.44250	0.68177	4.89511	0.37500
UM5	4.11450	3.96000	4.22750	4.49500	4.42500	4.24000	4.32250
UM6	0.50000	0.50000	0.50000	0.50000	0.50000	0.50001	0.50004
Num.	2	3	2	2	3	1	1

Table 18

Experimental results of the constrained multi-objective functions.

Fun.	DE/rand/1	DE/rand/2	DE/best/2	SCA	GWO	OFA	OFA/DP
M1	0.50000	0.50000	0.50000	0.50000	0.50021	0.50020	0.50007
M2	0.39084	0.39236	0.41557	0.39405	0.40931	0.37717	0.38307
M3	0.51692	0.59787	0.56780	0.59980	0.77007	0.80118	0.60881
M4	0.51098	0.50548	0.51240	0.50140	0.53366	0.50118	0.52067
M5	0.50377	0.51640	0.51451	0.50000	0.54857	0.78654	0.50250
M6	0.37917	0.39173	0.38146	0.46469	0.40855	0.37630	0.37517
M7	0.45455	0.46715	0.45639	0.60628	1.07806	0.83286	0.41829
M8	7.6E+07	8.0E+07	8.0E+07	0.34249	1.0E+07	2.0E+07	0.33340
M9	0.33838	0.33848	0.34359	0.33334	0.36192	0.33334	0.33334
M10	2.3E+07	2.3E+07	2.3E+07	1.9E+06	1.4E+07	1.0E+07	2.9E+06
Num.	1	0	0	4	0	2	3

$$\mathbf{x} = [x_1, x_2, \dots, x_D], x_i^L \leq x_i \leq x_i^U \quad (15)$$

where D is the quantity of decision variables, M is the quantity of objective functions, and $\mathbf{x}$ is the decision vector, $f(\mathbf{x})$ is the objective vector.

Many multi-objective optimization methods have been proposed in the literature, where the weighted sum method can transform the MOP into the single-objective optimization problem (SOP). The mathematical formula is described as follows:

$$\text{Minimize } wf(\mathbf{x}) = w_1f_1 + w_2f_2 + \dots + w_Mf_M \quad (16)$$

where $w = [w_1, w_2, \dots, w_M]^T$ is the weight vector, w_i usually set to $1/M$.

6.1. Unconstrained multi-objective functions

In this section, 6 unconstrained multi-objective functions are employed to verify the performance of the proposed algorithm. The detail description of these unconstrained multi-objective functions is shown in [31]. Each experiment was repeat 50 times with different random seeds, and the mean values of the optimization results are summarized in Table 17, the best results are typed in bold.

Table 19
Kruskal-Wallis ANOVA Table.

Source	SS	df	MS	Chi-sq	Prob>Chi-sq
Groups	3001.1	6	500.19	2.85	0.8275
Error	113923.9	105	1084.99		
Total	116925	111			

As shown in Table 17, GWO outperforms the other algorithms on UM1. OFA has the better result on UM3. OFA/DP possesses the best results on UM4. DE/rand/2 obtains the best results on UM5. For UM2 and UM6, DE/rand/1, DE/rand/2, DE/best/2 and SCA can converge to the optimum.

6.2. Constrained multi-objective functions

In this section, 10 constrained multi-objective functions are employed to verify the performance of the proposed algorithm. The detail definitions of these constrained multi-objective functions are shown in [32]. The constraint-handling technique of Section 5 is employed here to transform the constrained problems into unconstrained problems. Each experiment was repeat 50 times with different random seeds, and the mean values of the optimization results are summarized in Table 18, the best results are typed in bold.

From Table 18, it can be found that DE/rand/1 has the best results on M3, SCA possesses the best results on M1, M5, M9 and M10, OFA obtains the best results on M2 and M4, and OFA/DP get the best results on M6, M7 and M8.

6.3. Statistical analysis of the multi-objective functions

To analyze the performance of the algorithm on the multi-objective functions, the Kruskal-Wallis test was used to analyze the experimental result, and the statistical result is obtained, shown in Table 19. The p -value ($p = 0.8275$) is greater than 0.05, which shows that there is not an algorithm significantly better than others when handling the multi-objective functions.

7. The sensitivity of parameters n_{t_max} and r_c of OFA/DP

In this section, the sensitivities and interactions between n_{t_max} and r_c in OFA/DP are studied. Numerical experiments are carried out on 6 functions, they are the benchmark function F1, the CEC test function C1, the large scale function LS1, the CEC2017 constrained function CF1, the unconstrained multi-objective function UM1 and the constrained multi-objective function M1. In the experiment, n_{t_max} and r_c are set to two and three levels, i.e., $n_{t_max} \in \{5, 10\}$ and $r_c \in \{0.2, 0.5, 0.8\}$, respectively, and all possible combinations of each level are then run. Each experiment was repeat 50 times with different random seeds, and the mean values of the optimization results are summarized in Table 20, the best results are typed in bold.

As shown in Table 20, different parameters have little influences on F1, LS1, UM1 and M1. For the CEC2017 constrained function CF1, the results of the algorithm with the same n_{t_max}

and different r_c are quite different, but the results of the algorithm with different n_{t_max} and the same r_c are close. Hence, it can conclude that the parameter r_c has great influence on OFA/DP, the performance of OFA/DP decreases sharply with the increase of r_c .

8. Conclude

To extend the potentiality of OFA, a novel algorithm is proposed, named OFA/DP, which makes full use of the population information. During the iterations, OFA/DP divides the population into three states: evolution, exploration and stop, and different population states adopt different candidate generation strategies, they are the evolutionary direction prediction strategy, the Gaussian oscillation strategy and the OFA update method. The evolutionary direction prediction strategy predicts the evolution direction of the population by constructing the transition probability matrix, which improves the effective search of the algorithm; the Gaussian oscillation strategy explores the neighborhood space by means of Gaussian method, which enhances the ability of the algorithm to escape from the local optimum.

To verify the superiority of the proposed algorithm, OFA/DP is compared with DE/rand/1, DE/rand/2, DE/best/2, SCA, GWO and OFA on different kinds of functions, including 20 benchmark functions, 30 CEC test functions, 6 large scale functions, 28 CEC2017 constrained problems, 3 engineering problems, 6 unconstrained multi-objective functions and 10 constrained multi-objective functions. The principle of no-free-lunch shows that there is no an algorithm can solve all optimization problems well, and different algorithms have their own strengths. As shown in Section 4.2, GWO and SCA are good at solving the unimodal functions, DEs performs well on the multimodal functions, and the statistical results show that OFA/DP has the best integrative performance on the benchmark functions. Comparing with OFA, OFA/DP has better performance on most of the benchmark functions. However, the trade-off is that OFA/DP takes longer to run and consumes more computing resources. The statistical results of the experimental results on the CEC2017 constrained functions also show that OFA/DP is the best algorithm among the compared algorithms. Although the statistical results of the simulation results on the other test functions did not indicate the best algorithm, OFA/DP possesses huge advantages in dealing with these functions due to it can obtain the best results on most functions.

CRedit authorship contribution statement

ZhongQuan Jian: Conceptualization, Software, Writing - original draft, Formal analysis, Resources. **GuangYu Zhu:** Methodology, Funding acquisition, Project administration, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 20
Experimental results of OFA/DP with different parameters.

Fun.	Parameters $\{n_{t_max}, r_c\}$					
	{5, 0.2}	{5, 0.5}	{5, 0.8}	{10, 0.2}	{10, 0.5}	{10, 0.8}
F1	9.60e-320	6.52e-320	4.92e-320	1.11e-319	1.04e-319	9.07e-320
C1	-1399.733	-1399.686	-1399.638	-1399.721	-1399.713	-1399.656
LS1	2.57E+08	6.88E+08	3.44E+09	2.51E+08	5.60E+08	2.62E+09
CF1	1.92E-07	1.59E-04	3.73E+00	5.10E-08	1.27E-04	2.55E+00
UM1	0.375159	0.375082	0.375007	0.375153	0.375082	0.375007
M1	0.500060	0.500022	0.500000	0.500054	0.500023	0.500000

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.asoc.2021.107660>.

References

- [1] T. Bäck, H. Schwefel, An overview of evolutionary algorithms for parameter optimization, *Evol. Comput.* 1 (1) (2014) 1–23.
- [2] C.F. Wang, K. Liu, P.P. Shen, A novel genetic algorithm for global optimization, *Acta Math. Appl. Sin.* 36 (2) (2020) 482–491.
- [3] X.W. Xia, L. Gui, F. Yu, et al., Triple archives particle swarm optimization, *IEEE Trans. Cybern.* 50 (12) (2020) 4862–4875.
- [4] F. Tirado, R.J. Barrientos, P. González, et al., Efficient exploitation of the Xeon Phi architecture for the Ant Colony Optimization (ACO) metaheuristic, *J. Supercomput.* 73 (11) (2017) 5053–5070.
- [5] H. Ali, M.Z. Rafique, M.S. Sarfraz, et al., A Novel Approach for Solving Travelling Thief Problem using Enhanced Simulated Annealing, *PeerJ Comput. Sci.* 7 (1) (2021) e377.
- [6] X.S. Yang, A. Hossein Gandomi, Bat algorithm: a novel approach for global engineering optimization, *Eng. Comput.* 29 (5) (2012) 464–483.
- [7] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, *Adv. Eng. Softw.* 69 (2014) 46–61.
- [8] Y. Zhang, G. Dai, L. Peng, et al., Enhancing differential evolution algorithm through a population size adaptation strategy, *Nat. Comput.* (2021).
- [9] G.Y. Zhu, W.B. Zhang, Optimal foraging algorithm for global optimization, *Appl. Soft Comput.* 51 (2017) 294–313.
- [10] H. Liu, Z. Cai, Y. Wang, Hybridizing particle swarm optimization with differential evolution for constrained numerical and engineering optimization, *Appl. Soft Comput.* 10 (2) (2010) 629–640.
- [11] G.G. Wang, A.H. Gandomi, X. Zhao, et al., Hybridizing harmony search algorithm with cuckoo search for global numerical optimization, *Soft Comput.* 20 (1) (2016) 273–285.
- [12] R.M. Rizk-Allah, Hybridizing sine cosine algorithm with multi-orthogonal search strategy for engineering design problems, *J. Comput. Des. Eng.* 5 (2) (2018) 249–273.
- [13] C. Lu, L. Gao, J. Yi, Grey wolf optimizer with cellular topological structure, *Expert Syst. Appl.* 107 (2018) 89–114.
- [14] S. Asmussen, *Applied Probability and Queues*, Springer Science & Business Media, 2008.
- [15] J.R. Krebs, J.T. Erichsen, M.I. Webber, et al., Optimal prey selection in the great tit (*Parus major*), *Anim. Behav.* 25 (1977) 30–38.
- [16] P.A. Gagniuc, *Markov Chains: From Theory To Implementation and Experimentation*, John Wiley & Sons, 2017.
- [17] B.A. Craig, P.P. Sendi, Estimation of the transition matrix of a discrete-time Markov chain, *Health Econ.* 11 (1) (2002) 33–42.
- [18] M. Malik, L.C. Thomas, Transition matrix models of consumer credit ratings, *Int. J. Forecast.* 28 (1) (2012) 261–272.
- [19] T.W. Anderson, *An introduction to multivariate statistical analysis*, Wiley, New York, 1962.
- [20] T. Eltaieb, A. Mahmood, Differential evolution: A survey and analysis, *Appl. Sci.* 8 (10) (2018) 1945.
- [21] M. Jamil, X.S. Yang, A literature survey of benchmark functions for global optimization problems, *Int. J. Math. Model. Numer. Optimisation* 4 (2) (2013).
- [22] J.J. Liang, B.Y. Qu, P.N. Suganthan, et al., Problem Definitions and Evaluation Criteria for the CEC 2013 Special Session on Real-Parameter Optimization, Tech. Rep., Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, 2013.
- [23] J.J. Liang, B.Y. Qu, P.N. Suganthan, Problem Definitions and Evaluation Criteria for the CEC 2014 Special Session and Competition on Single Objective Real-Parameter Numerical Optimization, Tech. Rep., Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, 2013.
- [24] J.J. Liang, B.Y. Qu, P.N. Suganthan, et al., Problem Definitions and Evaluation Criteria for the CEC 2015 Competition on Learning-Based Real-Parameter Single Objective Optimization, Tech. Rep., Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, 2014.
- [25] N.H. Awad, M.Z. Ali, P.N. Suganthan, J.J. Liang, B.Y. Qu, Problem Definitions and Evaluation Criteria for the CEC 2017 Special Session and Competition on Single Objective Real-Parameter Numerical Optimization, Tech. Rep., Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, 2016.
- [26] K.V. Price, N.H. Awad, M.Z. Ali, P.N. Suganthan, Problem Definitions and Evaluation Criteria for the 100-Digit Challenge Special Session and Competition on Single Objective Numerical Optimization, Tech. Rep., Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, 2018.
- [27] X. Li, M. Tang, Z. Omidvar, K. Yang, K. Qin, Benchmark Functions for the CEC'2013 Special Session and Competition on Large Scale Global Optimization, Technical Report, Evolutionary Computation and Machine Learning Group, RMIT University, Australia, 2013.
- [28] E. Theodorsson-Norheim, Kruskal–Wallis test: BASIC computer program to perform nonparametric one-way analysis of variance and multiple comparisons on ranks of several independent samples, *Comput. Methods Programs Biomed.* 23 (1) (1986) 57–62.
- [29] D.H. Wolpert, W.G. Macready, No free lunch theorems for optimization, *IEEE Trans. Evol. Comput.* 1 (1) (1997) 67–82.
- [30] G.H. Wu, Mallipeddi, Rammohan, Suganthan, Ponnuthurai, Problem Definitions and Evaluation Criteria for the CEC 2017 Competition and Special Session on Constrained Single Objective Real-Parameter Optimization, Tech. Rep., Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, 2017.
- [31] E. Zitzler, K. Deb, L. Thiele, Comparison of multi-objective evolutionary algorithms: Empirical results, *Evol. Comput.* 8 (2000) 173–195.
- [32] Q.F. Zhang, A. Zhou, S.Z. Zhao, Suganthan, Ponnuthurai, W.D. Liu, S. Tiwari, Multi-objective optimization test instances for the CEC 2009 special session and competition, *Mech. Eng.* (2008).